Spark Out of Memory

class start @ 9:04 pm

1. Spark executor memory
2. Tungsten Project & Execution Engine
3. Data Frame caching & Persist [UDF]
4. Broadcasting in Join
5. Salting, Repartition and Coalesce

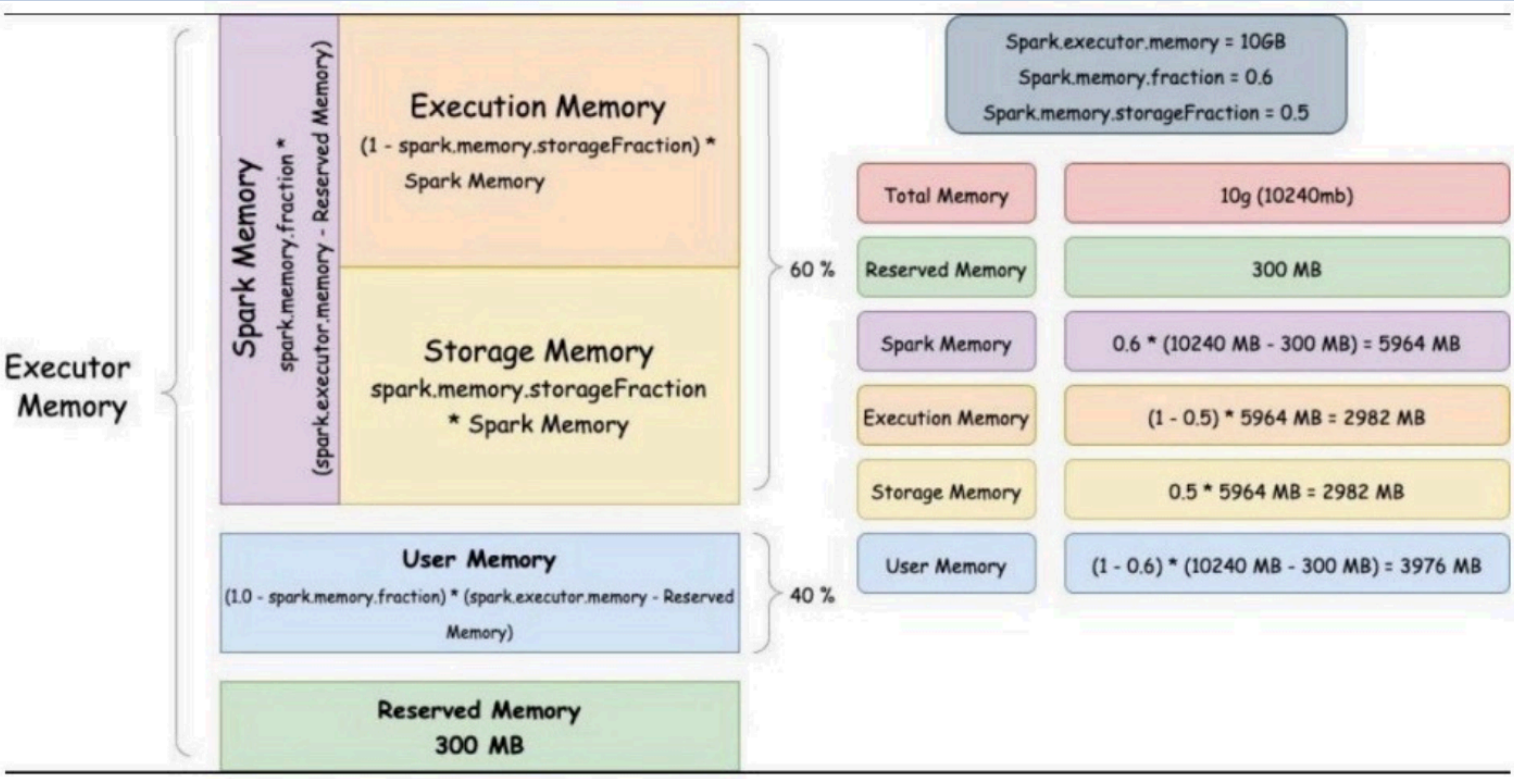
If time permits

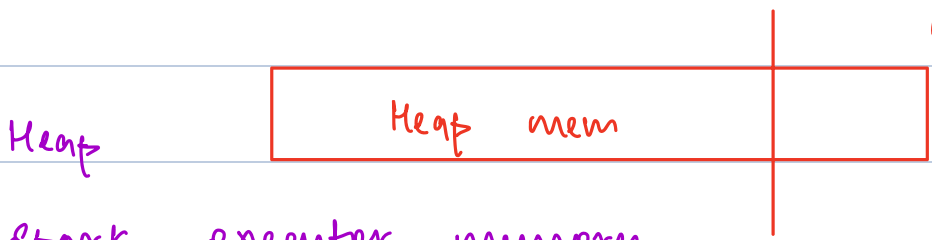
ADE

Spark Executor Memory [work horse]

Heap and Off-Heap Memory

advanced [Tungsten Engine]





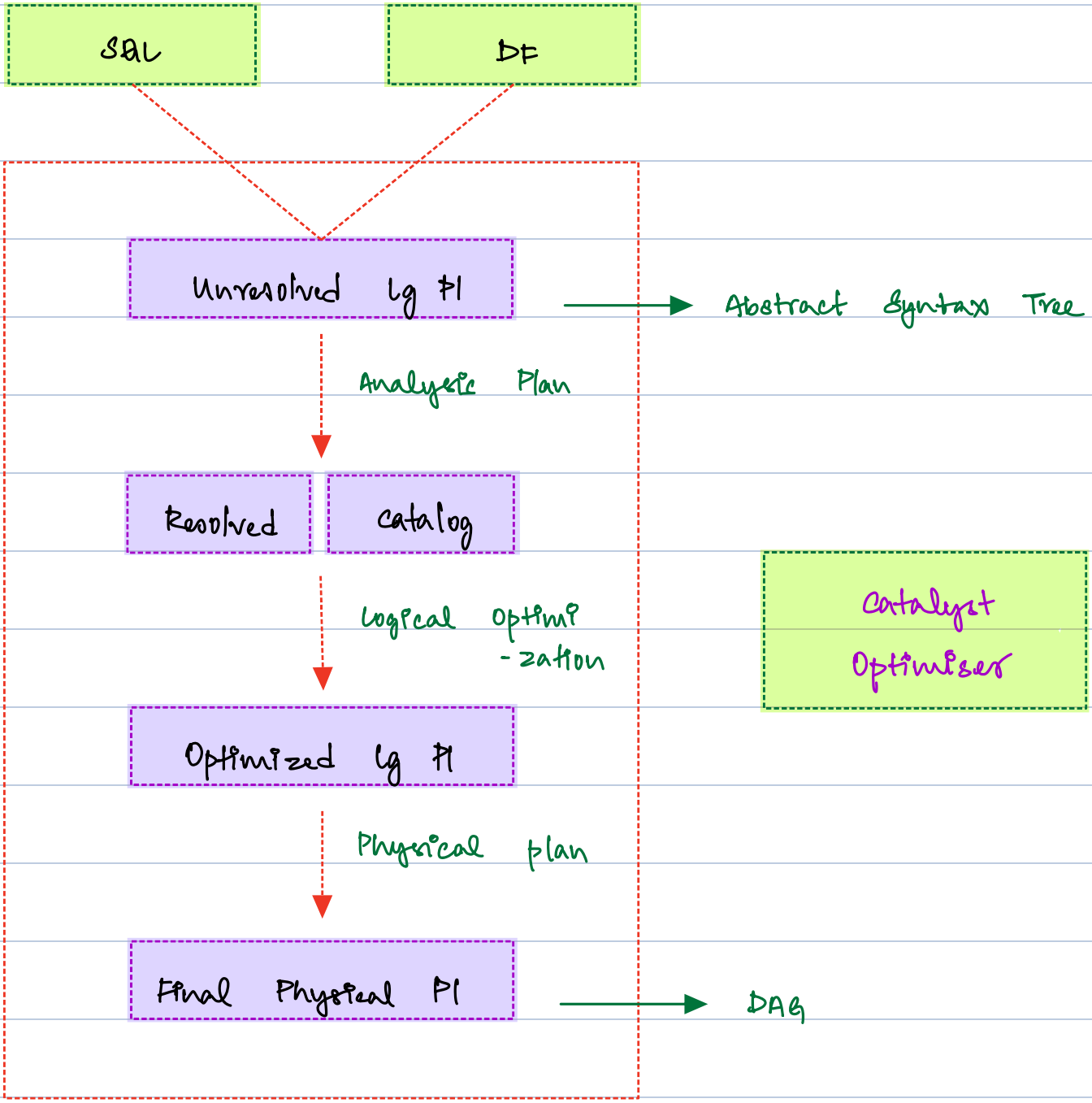
spark. executor. memory

4 different section

- Reserved memory
- user based memory
- spark memory
  - execution
  - storage

all of these are soft boundaries

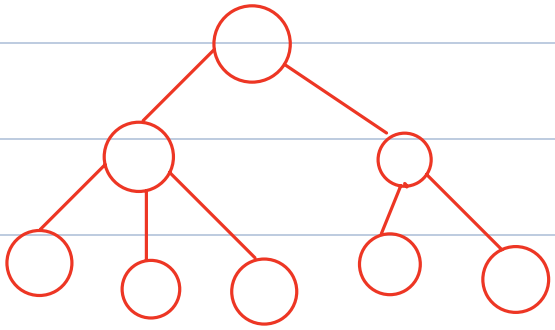
Execution Flow



Tungsten engine will take physical plan as input

Executes the plan

1 stage → most optimised Byte code WSCG



Stage → no shuffle

Final Physical PI

1. Whole-stage code Gen
2. Optimised Bytecode Generation
3. Unsafe Row & Off-Heap Memory
4. Avoid GC Overhead

Tungsten Engine

CPU registry optimised

10:02 pm → 10:10 pm

Break

Caching and Persisting

eliminate recomputing same data again and again

Spark gets DAG generated for any action

DF . count()

→ Scan , Filter , Project , count

everytime I run an action, it starts from scan, then performs all transformation from scratch and returns the data

cache and persist

more configurable

where you want to cache this data

persist stores the data in memory, memory AND DISK, OFF heap, using ser, or deser
local DISK avoiding GC

But cache is just on memory

Advanced Storage Types also involves Duplication of Data

Pro and Cons [Storage Level]

① Memory ⟶ Fastest cache, only store smaller DF

② Memory and Disk ⟶ middle ground, little slower

③ Disk ⟶ very large DF, slow

---

④ Off-Heap Faster, configuration decides the spa

spark.memory.offHeap.enabled

spark.memory.offHeap.size

UDF and Demo for cache and Persist

DF1

1
2
3
4

⟶

1	2
2	4
3	6
4	8

import UDF from pyspark.sql.functions

spark.udf.register("name", python fn, return Type)